Step 1: Market-data foundation (ticket PPA-1)

> **Time needed:** about 5 hours. Concepts take 90 minutes, running and reading the code 90 minutes, exercises and quiz 90 minutes, and your own to-dos 30 minutes. **You will be able to:** explain how German power prices are formed and published, handle time zones, daylight saving and mixed 60/15-minute data correctly, and show that a dataset is trustworthy before you build a model on it.

Contents: 0. Scenario · 1. Concepts · 2. Architecture · 3. Code tour · 4. Real problems · 5. Hands-on · 6. Results · 7. Exercises · 8. Quiz · 9. Interview · 10. Real work · 11. To-dos · 12. Glossary

---

0. The scenario

Imagine your first Monday in Encavis' Commercial team. Your lead writes:

> *"Welcome! Before you price anything we need a German price and generation dataset we can trust, going back to 2023, at the resolution the market actually trades. Please reconcile it against the official numbers and tell me what you find. Ticket is PPA-1."*

That is how quant work really starts. Every PPA price, hedge ratio and risk figure later in this project is computed from these prices and volumes. If the data is shifted by one hour, solar output is matched with the wrong prices and the capture price can be wrong by several EUR/MWh. On a 10-year, 200 GWh-per-year PPA, 3 EUR/MWh is **EUR 6 million**. So Step 1 is about trust.

Ticket PPA-1: acceptance criteria (all met today)

- ☑ Reproducible repository: package, configuration, tests, CI, documentation.
- ☑ Day-ahead prices (DE-LU) and generation by technology, Jan 2023 to Sep 2026, at native resolution, cached locally with provenance.
- ☑ Correct handling of daylight saving and of the 15-minute switch on 1 Oct 2025.
- ☑ Automated data-quality checks that fail loudly.
- ☑ Reconciliation against published figures (average price, negative hours).
- ☑ A short report with charts and a written interpretation.

---

1. Concepts you need today

1.1 How a power price is formed: the merit order

Electricity is bought and sold in an **auction** for every delivery interval of tomorrow. Generators offer at roughly their **marginal cost**, the cost of producing one more MWh. Wind and solar have almost no fuel cost, so they bid near zero. Lignite, hard coal and gas follow, then expensive peaking plants. Sorting the

offers from cheap to expensive gives the **merit order**. The **clearing price** is the offer of the last plant needed to meet demand, and **every** accepted seller receives it (uniform pricing).

A toy example with 55 GW of demand:

Supply block	Capacity (GW)	Offer (EUR/MWh)	Cumulative (GW)
Wind + solar (cloudy, calm day)	25	0	25
Lignite	10	40	35
Hard coal	8	70	43
Gas (CCGT)	15	110	58

Demand of 55 GW is met inside the gas block, so **the price is 110 EUR/MWh**.

Now make it sunny and windy, so wind + solar offer 45 GW at 0. The cumulative supply reaches 55 GW inside the lignite block, so **the price falls to 40 EUR/MWh**. If renewables alone exceed demand, the price can fall to zero or below.

This is the **merit-order effect**. When renewables produce a lot, they push the price down, *exactly in the hours they produce*. That is why solar earns less than the average price, which is called **cannibalisation**. Step 2 quantifies it.

1.2 Why prices go negative

A negative price means a producer **pays** to deliver power. Producers accept that when stopping costs them more than paying:

- **Inflexible plants**: lignite and nuclear have minimum load levels and costly restarts; combined heat and power plants must keep running to deliver heat.
- **Subsidies paid per MWh**: older plants with fixed feed-in tariffs or market premiums kept earning support even at negative prices, so they bid below zero. Older EEG rules removed support only after several consecutive negative hours (six hours for plants commissioned from 2016; shortened in later EEG versions). For plants commissioned since 25 Feb 2025 (Solarspitzengesetz) support stops in every negative-price period.
- **Limited flexibility**: storage, flexible demand and export capacity are not yet large enough to absorb midday solar peaks.

For Encavis this matters directly. Its 2024 annual report says that parks in Spain and Finland were shut down for the first time because of negative prices, and that revenue suffered as a result.

1.3 Markets and products

Market	What is traded	When	Who runs it	Role in our project
Futures (EEX)	Phelix DE Base / Peak for months, quarters, years	Years to days ahead	EEX	Hedging (Step 8)
Day-ahead auction (SDAC)	Each 15-minute interval of tomorrow (hourly before 1 Oct 2025)	Bids by 12:00 CET, results early afternoon	Power exchanges such as EPEX SPOT, coupled across Europe	**Our price data today**
Intraday	Continuous trading and auctions for today and tomorrow	Up to minutes before delivery	EPEX SPOT and others	Correcting forecast errors
Balancing	Grid operators settle deviations (reBAP price)	After delivery	German TSOs	Cost of being wrong on volume

Key facts behind today's data:

- **Bidding zone DE-LU.** Germany and Luxembourg share one price. Austria was split off on 1 Oct 2018.
- **15-minute products since 1 Oct 2025.** The European day-ahead market (SDAC) moved from 24 hourly prices to **96 quarter-hourly prices per day**. One price series therefore contains two resolutions, and our code must handle both.
- **Price limits.** The harmonised maximum is +4,000 EUR/MWh. The minimum was −500 EUR/MWh and **became −600 EUR/MWh for deliveries from 29 May 2026**, because prices came close to the old floor often enough to trigger the automatic adjustment rule. A price outside the limits cannot be a real auction result. If we see one, our data or parsing is broken, so it is a perfect data-quality check.

1.4 Units: the classic sources of factor-of-10 errors

Quantity	Unit	Conversion
Price	EUR/MWh (markets) or ct/kWh (German tariffs, market values)	1 ct/kWh = 10 EUR/MWh
Power	MW (a rate, at one moment)	
Energy	MWh (an amount, over time)	MWh = average MW x hours
15-minute energy		MWh = MW x 0.25

> **Pitfall:** generation data arrives in **MW** (average power per interval). Summing 15-minute MW values without multiplying by 0.25 overstates energy fourfold.

1.5 Time: the most common bug in energy data

Power is delivered in **local time** (Europe/Berlin), which has daylight saving:

Year	Spring (23-hour day)	Autumn (25-hour day)
2023	26 March	29 October
2024	31 March	27 October
2025	30 March	26 October
2026	29 March	25 October

On the spring day the hour 02:00 to 03:00 does not exist; on the autumn day 02:00 to 03:00 happens twice. A delivery day can therefore have 23, 24 or 25 hours, i.e. 92, 96 or 100 quarter-hours. Our rules (ADR 0002):

1. **Store in UTC**, which never jumps.
2. A timestamp is the **start** of its interval ("interval beginning").
3. Convert to Berlin time only to group by delivery day or month, or to display.

> **Pitfall:** in plain Python, subtracting two `datetime` objects that share the same time-zone object gives the wall-clock difference. Midnight 30 March to midnight 31 March comes out as 24 hours, but it was 23. The test `test_plain_python_datetime_gets_dst_wrong` documents this; the project uses pandas timestamps, which compute in absolute time.

1.6 The data-source landscape

Source	Content	Access	Typical use
EPEX SPOT / EEX	Official auction and futures results	Paid licence	The "golden source" in companies
ENTSO-E Transparency Platform	Prices, load, generation for all of Europe	Free, needs a token	Second source, other countries
SMARD.de (Bundesnetzagentur)	German prices and generation	Free downloads	Upstream of our price data
Energy-Charts API (Fraunhofer ISE)	Prices (from SMARD) and generation, JSON	**Free, no key, CC BY 4.0**	**Our primary source today**
netztransparenz.de (TSOs)	Official monthly and annual market values	Free	Reconciliation (Step 2)

Licence matters. CC BY 4.0 allows reuse, including commercial use, *with attribution*. Every report this project produces says "CC BY 4.0, Bundesnetzagentur | SMARD.de, via Energy-Charts (Fraunhofer ISE)". In a company, data licences are contracts; using vendor data outside its licence is a compliance issue.

---

2. What was built: architecture

```
Energy-Charts API (JSON over HTTPS)
      |  EnergyChartsClient: timeouts, retries, Retry-After
      |  parse_price_payload / parse_public_power_payload (pure functions)
      v
RAW        data/raw/energy_charts/<dataset>/<YYYY-MM>.parquet + .meta.json
           exactly what the API returned; never edited; cached
      |  build_processed()
      v
PROCESSED  data/processed/: prices_native, prices_hourly, power_15min, power_hourly
      |
      +--> quality.run_all()             --> reports/data_quality.md
      +--> reconcile + step01_report      --> reports/step01_market_data.md + figures/
```

The repository:

```
ppa-lab/
  config/project.toml          every assumption in one place
  src/ppa_lab/
    config.py                  typed, validated settings
    timeutils.py               UTC, DST, interval lengths, hourly views
    data/energy_charts.py      API client + parsers
    data/store.py              raw cache with manifests
    data/pipeline.py           fetch months, build processed data
    data/quality.py            data-quality checks
    analysis/reconcile.py      statistics + tie-out
    analysis/step01_report.py   report and charts
    cli.py                     the `ppa-lab` command
  tests/                       52 offline tests + 1 live contract test
  docs/decisions/              ADRs 0001-0003
  docs/learning_track/         this page
  reports/                     generated output
  Makefile, pyproject.toml, .github/workflows/ci.yml, CHANGELOG.md, LICENSE
```

---

3. Code tour: every file, and why it looks the way it does

For each part: **what** it does, **why** it is built that way, and the **real-world** equivalent.

3.1 Repository and tooling

- **`src/` layout + `pyproject.toml`.** The package is installed in editable mode (`pip install -e .`), so tests import exactly what users import. `pyproject.toml` declares dependencies, the `ppa-lab` command, and the pytest and ruff settings in one standard file.
- **`Makefile`.** One-word commands (`make test`, `make all`) that a colleague or the CI server can run without reading your mind.
- **`.gitignore`.** Keeps data (rebuildable), the virtual environment and secrets (`.env`) out of Git.

- `.env.example`. Shows which secrets exist (the ENTSO-E token) without containing them.
- **CI (`.github/workflows/ci.yml`).** Every push runs lint and tests on a clean machine. Because tests are offline, CI never fails just because an API is down.
- **ADRs (`docs/decisions/`).** Short records of *why* a design was chosen. Six months later, nobody remembers why timestamps are UTC; the ADR does.
- `CHANGELOG.md` **and** `docs/BACKLOG.md`. What changed, and what is next, written as tickets with acceptance criteria.

> **Real-world note:** a commercial team's analytics code lives in the same kind of repository, reviewed through merge requests and deployed by IT. Showing this structure in your portfolio signals "can work in our codebase from week one".

3.2 `config/project.toml` and `config.py`: one source of truth

All assumptions sit in one TOML file: bidding zone, date range, price limits with their effective dates, API settings, and the published reference values. `config.py` turns it into frozen dataclasses and validates it (for example, start must not be after end, and min must be below max).

The price limits are a **time-dependent table**, not a single number:

```toml
[[market.price_limits]]
effective_from = 2023-01-01
min_eur_mwh = -500.0
max_eur_mwh = 4000.0

[[market.price_limits]]
effective_from = 2026-05-29
min_eur_mwh = -600.0
max_eur_mwh = 4000.0
```

`Settings.price_limit_on(day)` returns the limit in force on a given delivery day.

> **Why:** market rules change. If the floor were hard-coded as –500, a perfectly real –550 EUR/MWh price in June 2026 would be flagged as a data error.

3.3 `timeutils.py`: time done right

Key functions:

- `hours_in_local_day(day, tz)` returns 23, 24 or 25.
- `expected_intervals(day, tz, resolution_min)` returns 92/96/100 at 15 minutes, or 23/24/25 at 60 minutes.
- `interval_minutes(index)` gives the length of each interval. It takes the minimum of the distance to the previous and to the next timestamp:

Timestamp (UTC)	Previous gap	Next gap	Interval
30 Sep 21:00 (last hourly price)	60	60	**60**
30 Sep 22:00 (= 1 Oct 00:00 Berlin, first quarter-hour)	60	15	**15**
30 Sep 22:15	15	15	**15**

This also survives gaps: a missing hour does not turn its neighbour into a 2-hour interval. - `to_hourly(frame)` averages sub-hourly values into hourly values and leaves empty hours as NaN, so gaps stay visible. - `normalise_index(index)` makes every index UTC, nanosecond resolution, named `ts_utc` (see the pandas 3 story in section 4).

3.4 `data/energy_charts.py`: a robust API client

Two halves:

1. **Pure parsers** (`parse_price_payload`, `parse_public_power_payload`). They check that the expected keys exist, that the lengths match, and that the unit is "EUR / MWh", then build a DataFrame. No network, so they are easy to test with saved responses.
2. **The client** (`EnergyChartsClient.get_json`). It sets a timeout (never wait forever), a User-Agent (identify yourself to the provider), and a retry policy:

Situation	Action
HTTP 200	Return the JSON
HTTP 429 (rate limited) with `Retry-After: 19`	Wait 19 + 1 s, retry
HTTP 429/500/502/503/504 without the header	Exponential back-off: 1, 2, 4, 8 ... s
Connection error or timeout	Exponential back-off
Other 4xx (e.g. 400 bad parameter)	Fail immediately: retrying will not fix our request
Still failing after `max_retries`	Raise `EnergyChartsError` with the URL and parameters

`sleep` is an injectable field, so tests check the waiting logic without actually waiting.

> **Good practice:** the `user_agent` names the project and its home (`github.com/ankitpatel1661/ppa-lab`), so the data provider knows who is calling.

3.5 `data/store.py`: the raw layer and its manifests

Each month is saved as `data/raw/energy_charts/<dataset>/<YYYY-MM>.parquet` with a `.meta.json` next to it. For example, `prices/2025-10.meta.json` contains:

```
{
  "source_url": "https://api.energy-charts.info/price",
  "params": {"bzn": "DE-LU", "start": "2025-10-01", "end": "2025-10-31"},
  "complete": true,
  "license": "CC BY 4.0 – Bundesnetzagentur | SMARD.de, via Energy-Charts (Fraunhofer ISE)",
  "rows": 2980,
  "sha256": "...",
  "written_at_utc": "2026-10-07T12:3x:xx+00:00"
}
```


(October 2025 has 2,980 rows: 31 days x 96 quarter-hours + 4 extra for the 25-hour day.)

Design choices:

- **Atomic writes.** Data goes to a temporary file, then `os.replace` renames it, so a crash never leaves a half-written file.
- **Immutable by default.** `save()` refuses to overwrite unless explicitly asked.
- **Provenance.** Parameters, licence, checksum and timestamp answer the auditor's question "where does this number come from?".
- **`complete` flag.** A month downloaded while still in progress is fetched again next time.

3.6 `data/pipeline.py`: fetch and build

- `month_chunks(start, end)` splits the range into calendar months, handling month lengths and leap years (February 2024 has 29 days, and a test checks it). Monthly chunks keep requests small and make the cache granular.
- `fetch()` downloads only what is missing or incomplete, pausing between requests to respect the provider.
- `build_processed()` creates four datasets:

File	Content
`prices_native.parquet`	Every price at its native resolution + `interval_min`
`prices_hourly.parquet`	Hourly average + `n_intervals` (1 before Oct 2025, 4 after)
`power_15min.parquet`	Generation by technology in MW, 15 minutes
`power_hourly.parquet`	Hourly average MW = MWh produced in that hour

3.7 `data/quality.py`: checks that fail loudly

Check	What bug it catches
Index is UTC, sorted, no duplicates	Mixed time zones, double-loaded files
No missing prices	Silent gaps that bias averages
Within SDAC limits (date-dependent)	Parsing errors, unit mix-ups (ct vs EUR)
Intervals at the price floor (warning)	Real but extreme events worth reviewing
Complete delivery days, DST-aware	Lost or duplicated hours around clock changes
Covers the configured range	A month that silently failed to download
Resolution regimes (information)	Documents the 60 to 15-minute switch
Required generation columns present	API changes that drop a series
Missing-value share per column	Gaps in generation data
Solar non-negative	Sign errors
No solar at night (00:00-03:00 local)	**Time-zone shifts.** If solar shows up at midnight, timestamps are wrong

The night-solar check is a **physics check**. A test (`test_time_zone_bug_is_caught_by_night_solar_check`) shifts the data by 12 hours and confirms the check fails.

3.8 `analysis/reconcile.py`: statistics and the tie-out

`annual_price_stats` computes per local year:

- **Time-weighted mean price.** Each price is weighted by its interval length, because since Oct 2025 a row can be 15 or 60 minutes. A plain row average would give a quarter-hour four times the weight of an hour.
- **Three definitions of "negative hours"**, because publishers differ:
- `negative_hours`: the hourly average price is below zero (the classic definition);
- `hours_with_any_negative_interval`: at least one negative quarter-hour in the hour;
- `negative_time_hours`: the total duration of negative intervals.
- `longest_negative_streak_h`, which links to the EEG rules above.

`reconcile()` compares our numbers with the published references from the config and reports the difference and whether it is within tolerance.

> **Real-world note:** "reconcile to an independent source" is standard practice in risk and finance teams. A new model, curve or data feed is not used until it ties out. Saying this in an interview signals maturity.

3.9 `analysis/step01_report.py` and `cli.py`

The report module writes `reports/step01_market_data.md`, `reports/data_quality.md` and four charts. Chart rules used here: one y-axis per chart, colour by series, direct labels where helpful, light grid.

`cli.py` exposes everything as `ppa-lab fetch | build | check | report | all`, and `check` exits with code 1 on failure, so automation can stop a broken pipeline.

3.10 Tests: what each file protects

Test file	Protects against
`test_timeutils.py`	DST arithmetic, interval lengths at the resolution switch and with gaps
`test_energy_charts.py`	Parser contract (keys, units, lengths), retry policy, `Retry-After`, query parameters
`test_store.py`	Lossless save/load, checksums, accidental overwrite
`test_pipeline.py`	Month chunking (leap year), caching, re-download of incomplete months, processed outputs
`test_quality.py`	Each check fails on the bug it targets; the −600 floor from 29 May 2026
`test_reconcile.py`	Time-weighted mean across resolutions; the three negative-hour definitions
`test_report.py`	The whole chain end to end on two real days
`test_cli.py`	Commands and exit codes: `check` returns 1 on bad data, so automation stops
`test_config.py`	Config loads and validates

The fixtures are **real API responses** chosen for their difficulty: the switch day, both DST days, and generation on the spring DST day. A "fake client" stands in for the network. One test is marked `network` and runs only with `pytest -m network`. It is a **contract test** that warns you if the live API changes.

---

4. Two real problems hit today, and how they were debugged

These really happened while building Step 1. Interviewers love these stories, so learn them.

4.1 pandas 3 changed how timestamps are stored

- **Symptom:** after installing the project, 9 tests failed; interval lengths came out as `0.06` instead of `60` minutes.
- **Investigation:** `interval_minutes` used `index.asi8`, the raw integer behind each timestamp, and assumed nanoseconds. The new virtual environment had installed **pandas 3.0**, which stores datetimes in the unit of the input (here *seconds*), so the integers were 10^9 times smaller than expected.
- **Fix:** compute differences as time spans (`(t2 - t1) / pd.Timedelta(minutes=1)`) instead of raw integers.
- **Second effect:** Parquet cannot store second-precision timestamps, so data came back in milliseconds and the save/load test failed. Fix: normalise every index to one unit (`TIME_UNIT = "ns"`) where data enters the system and where it is loaded.
- **Lesson:** major library versions change behaviour. Tests caught it within a second, not in a board memo. In a company you would also **pin** versions in a lock file.

4.2 The API rate-limited us (HTTP 429)

- **Symptom:** the first full download stopped at June 2023: "Giving up after 4 attempts".
- **Evidence first, fix second:** a manual probe showed that the API allows a few quick requests, then answers `429 Too Many Requests` **with a** `Retry-After: 19` **header**. The original back-off (1, 2, 4 s) was simply too short.
- **Fix, test-first:** a new test (`test_client_honours_retry_after_on_rate_limit`) was written and failed (red); then the client was changed to obey `Retry-After` (green). The pause between requests also went from 0.4 s to 1.5 s.
- **Lesson:** read what the server tells you before guessing. Write the failing test before the fix, so the bug can never silently return.

A third, smaller one: the report crashed when the configured range did not contain the 15-minute switch (no data to plot). Now charts without data are skipped, and an end-to-end test covers it.

---

5. Hands-on: run it yourself

From the `ppa-lab` folder:

```
make setup                # 1. create .venv and install (once)
make test                 # 2. 52 passed, 1 deselected, in about 1-2 seconds
make all                  # 3. fetch (10-15 min first time) -> build -> check -> report
open reports/step01_market_data.md
```

Explore the data yourself in Python (`.venv/bin/python`):

```
import pandas as pd
prices = pd.read_parquet("data/processed/prices_native.parquet")
berlin = prices.tz_convert("Europe/Berlin")
berlin.loc["2025-10-26"].shape          # (100, 2): the 25-hour day at 15 minutes
berlin["price_eur_mwh"].idxmin()        # when was the lowest price?
hourly = pd.read_parquet("data/processed/prices_hourly.parquet")
hourly["n_intervals"].value_counts()    # 1 = hourly era, 4 = quarter-hour era
```

Look inside a manifest: `cat data/raw/energy_charts/prices/2025-10.meta.json`.

---

6. Results: what the data says

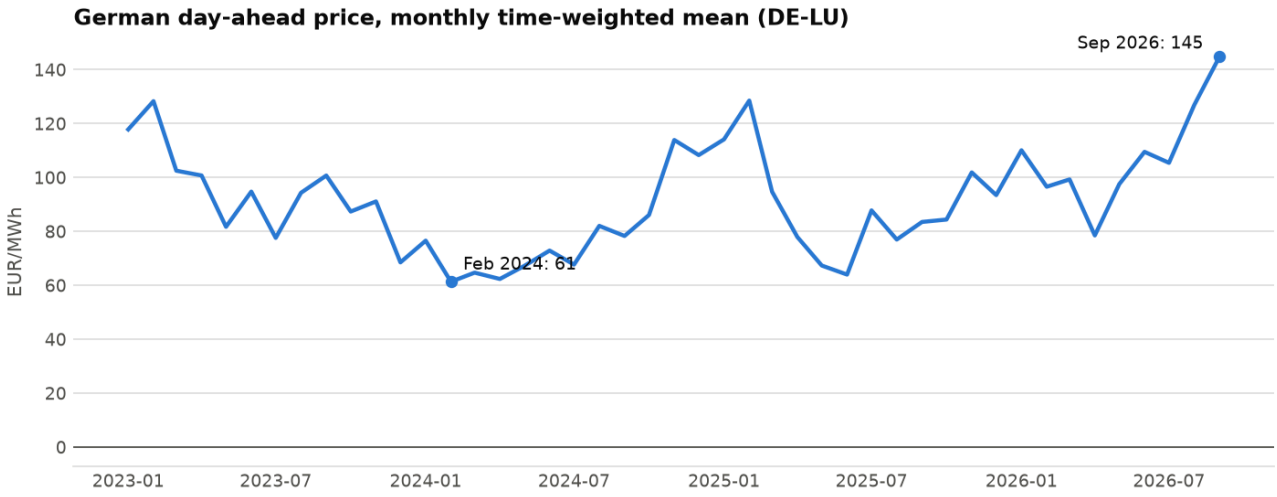
6.1 The tie-out: the dataset is trustworthy

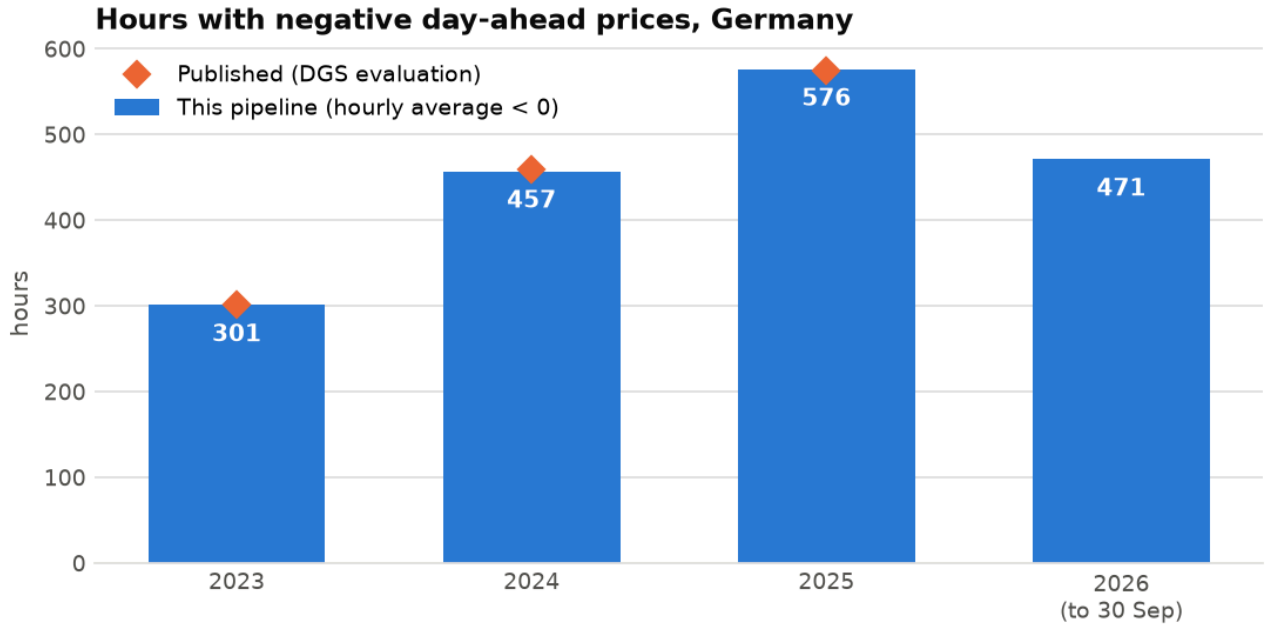
Check	This pipeline	Published	Verdict
2025 average day-ahead price	**89.32 EUR/MWh**	8.932 ct/kWh = 89.32 EUR/MWh (netztransparenz, via DGS)	Exact match
Negative hours 2023	**301**	301 (DGS)	Exact match
Negative hours 2024	**457**	459 (DGS); 457 (other sources)	Matches 457; within tolerance of 459
Negative hours 2025	**576**	575 (DGS); 573 (pv magazine)	Within tolerance; see below
Data quality	23 pass, 1 warn, 0 fail		All 1,369 delivery days complete

The 2025 difference is a lesson in definitions. Since 1 Oct 2025 an hour can be partly negative. Our three measures for 2025 are 576 hours with a negative *hourly average*, 588 hours with *at least one* negative quarter-hour, and **574.75 hours of negative time**, which rounds to the published 575. The publisher apparently counts duration. Whenever two sources disagree slightly, check the definition before suspecting the data.

6.2 The annual picture

Year	Mean (EUR/MWh)	Lowest price	Highest price	Negative hours	Longest negative streak
2023	95.18	−500.00, Sun 2 Jul 14:00	524.27, Mon 11 Sep 19:00	301	36 h, 24-25 Dec
2024	78.51	−135.45, Sun 12 May 13:00	936.28, Thu 12 Dec 17:00	457	18 h, Sun 7 Jul
2025	89.32	−250.32, Sun 11 May 13:00	583.40, Mon 20 Jan 17:00	576	20 h, 4-5 Oct
2026 (Jan-Sep)	107.72	−499.99, Fri 1 May 13:15	747.10, Wed 24 Jun 20:45	471	18 h, Easter Sunday 5 Apr



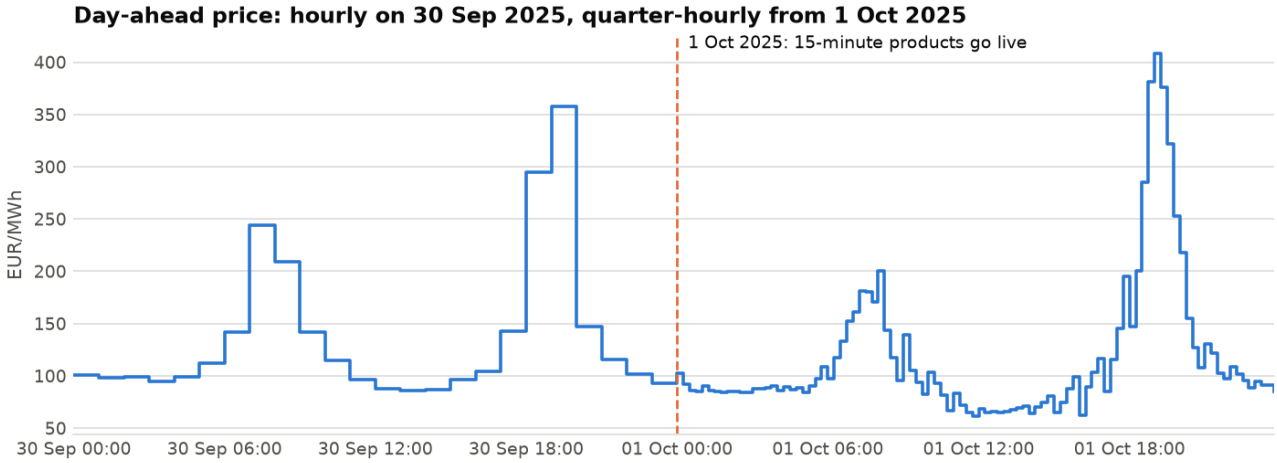


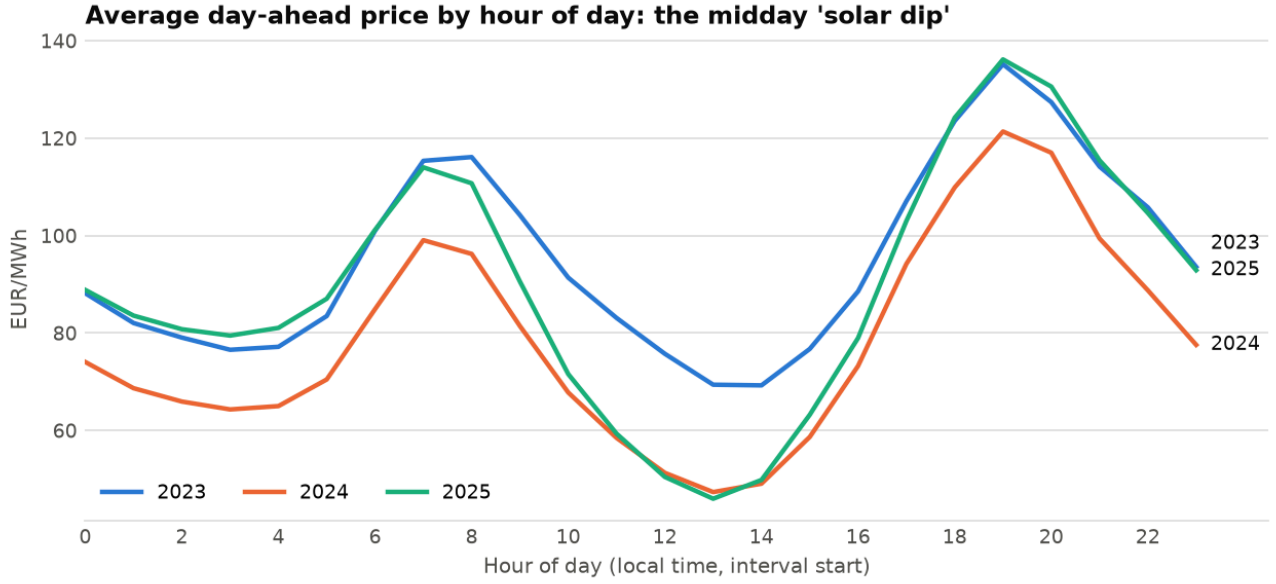
6.3 Six stories hidden in the data

1. **The merit order, live.** On Sunday 11 May 2025 at 13:00, solar alone produced 42.1 GW while load was only 40.1 GW, and the price fell to −250 EUR/MWh. On Monday 20 January 2025 at 17:00 there was no sun, only 4.5 GW of wind and 70.6 GW of load, so expensive plants set the price at 583 EUR/MWh. The 2024 maximum of 936 EUR/MWh on 12 December 2024 was a *Dunkelflaute* (dark, windless winter evening: 29 MW of solar and 1.3 GW of wind against 68 GW of load).
2. **The floor was hit, and then it moved.** The −500 EUR/MWh floor was reached once, on Sunday 2 July 2023 at 14:00. Our quality report flags it as a warning: real, but worth a look. In spring 2026, prices fell to −480.01 (Sunday 26 April) and −499.99 EUR/MWh (Friday 1 May, a public holiday). That means prices below 70% of the floor on two different days within 30 days, which is the pattern that triggers the harmonised adjustment rule. The floor became −600 EUR/MWh from 29 May 2026. You can see a market rule change being *caused* in your own data.
3. **The midday dip is deepening.** The average price at 13:00 fell from 69 EUR/MWh (2023) to 47 (2024) and 46 (2025), while 19:00 stays around 135 EUR/MWh. That evening-minus-midday

spread of about 90 EUR/MWh is bad news for solar (Step 2: capture rates) and good news for batteries (Satellite A: tolling).

4. **The peak premium has almost gone.** In 2023 the EEX peak window (weekdays 08-20) averaged 106.24 vs a base of 95.18 EUR/MWh, a premium of 11 EUR/MWh. In 2025 it was 92.35 vs 89.32, only 3 EUR/MWh, because midday solar now sits inside the peak window.
5. **Negative prices are seasonal.** In 2025 there were 129 negative hours in May and 141 in June, but none in November and December. April 2026 alone had 123. Spring (strong sun, mild demand, holidays) is the danger zone for solar revenue.
6. **Quarter-hours reveal hidden volatility.** In the first year of 15-minute prices (Oct 2025 to Sep 2026) the median spread between the highest and lowest quarter-hour *within the same hour* was 15 EUR/MWh; the 95th percentile was 76 EUR/MWh and the maximum 460 EUR/MWh (24 June 2026, 19:00). An hourly dataset would hide all of this. There were 2,052 negative quarter-hours (513 hours' worth).





Live pitfall, hit while writing this page: flooring Berlin-time timestamps to the hour crashed with `Cannot infer dst time from 2025-10-26 02:00:00`, because 02:00 happened twice that night. The same operation in UTC works. That is ADR 0002 in action.

6.4 What this means for pricing a solar PPA

- Volumes and prices are **negatively correlated**: the more solar Germany produces, the lower the price in exactly those hours. A PPA price based on the *average* (baseload) price would overpay for solar output, by a lot.
- Negative hours cluster in sunny spring and summer middays, so contract clauses about negative prices (who pays, whether to curtail) are worth real money.
- Prices in 2026 are higher than in 2025 (mean 107.72 vs 89.32 EUR/MWh, with September 2026 at about 145 EUR/MWh). The *level* moves with fuel and CO2 prices, while the *shape* (midday dip) keeps deepening. Pricing models must treat level and shape separately; Step 5 does exactly that.
- **Next (Step 2, PPA-2):** compute what solar and wind actually earn, the capture price and capture rate, and reconcile it with the official "Marktwert Solar".

---

7. Exercises

Try each one before opening the hints. Write a test first where it says so: that is the habit employers look for.

Warm-up

1. **Peak vs base.** EEX defines *Peak* as Monday to Friday, 08:00-20:00 local time. Compute the 2025 average peak and off-peak price. Which is higher, and why has the gap narrowed in recent years?
2. **The extreme hour.** Find the lowest and highest hourly price of 2025. On which day and hour did each occur? Look at solar, wind and load in `power_hourly.parquet` at that time and explain the price with the merit order.
3. **Units drill.** Using `power_15min.parquet`, compute total German solar generation in 2025 in TWh. (Remember: MW x 0.25 h per quarter-hour; 1 TWh = 1,000,000 MWh.)

Core

4. **New DQ check (test first).** Add a check "no stale prices": fail if the price is identical for 24 or more consecutive hours (a frozen feed). Write the failing test in `tests/test_quality.py`, then implement it in `quality.py`.
5. **Residual load vs price.** Residual load = load minus wind minus solar. Plot hourly price against residual load for 2025 (scatter). Describe the shape: this is the merit order, seen in real data.
6. **Definitions matter.** Since 1 Oct 2025, compare `negative_hours`, `hours_with_any_negative_interval` and `negative_time_hours`. By how much do they differ, and which would you use in a PPA clause, and why?

Stretch

7. **Second source.** When your ENTSO-E token arrives, write `data/entsoe.py` (the `entsoe-py` package helps), download one month of DE-LU prices and reconcile them interval by interval with Energy-Charts.
8. **Another market.** Make a copy of the config for France (`bidding_zone = "FR"`, `country = "fr"`, separate `paths`) and run the pipeline with `ppa-lab --config config/fr.toml all`. Which code

had to change? (Ideally none.)

▼ **Hints and solution sketches**

1. `h = hourly.tz_convert("Europe/Berlin"); peak = (h.index.dayofweek < 5) & (h.index.hour >= 8) & (h.index.hour < 20)`; then average `price_eur_mwh` for `peak` and `~peak` in 2025. Midday solar now depresses the peak window, so the peak premium is smaller than it used to be.
2. `hourly.loc["2025", "price_eur_mwh"].idxmin()`; the lowest prices fall on sunny weekend or holiday middays (high solar, low load); the highest on calm, dark winter evenings (low wind, no solar, high load, gas sets the price).
3. `p = pd.read_parquet("data/processed/power_15min.parquet").tz_convert("Europe/Berlin"); (p.loc["2025", "solar_mw"] * 0.25).sum() / 1e6`. You should get about 70.1 TWh (2023: 53.9 TWh, 2024: 59.7 TWh): public net solar generation.
4. Use `longest_run(prices["price_eur_mwh"].diff().eq(0))` on hourly data; status "fail" if the run is 24 or more.
5. `r = power_hourly["load_mw"] - power_hourly[["wind_onshore_mw", "wind_offshore_mw", "solar_mw"]].sum(axis=1)`, join with hourly prices, `plt.scatter(r, price, s=1, alpha=0.2)`. You will see a rising, convex curve: low or negative prices at low residual load, steep at high.
6. Read the three columns from `annual_price_stats` for 2025-10-01 onwards. Contracts must define it precisely, typically using the settlement interval of the market (now 15 minutes).
7. Use `EntsoePandasClient(api_key=...).query_day_ahead_prices("DE_LU", start=..., end=...)`; differences should be zero or rounding only.
8. Only the config. That is the payoff of keeping assumptions out of code.

---

8. Self-check quiz

1. Why does a sunny day lower the price for *all* producers, not only solar?
2. How many quarter-hour prices does 26 October 2025 have, and why?
3. What changed on 1 October 2025, and what does it do to a plain average of rows?
4. Why do we store timestamps in UTC and convert only for grouping and display?
5. A price of –550 EUR/MWh appears on 28 May 2026. Data error or not? And on 29 May 2026?
6. What does a `Retry-After: 19` header mean, and how does our client react?
7. Why are raw files never edited, and where do corrections happen instead?
8. Which check would catch a one-hour time-zone shift in generation data?
9. Give two definitions of "negative hours" and explain when they differ.
10. What is a reconciliation, and what do you do if it fails?

▼ **Answers**

1. Uniform pricing: the clearing price is set by the marginal plant and paid to all; more zero-cost supply moves the intersection to a cheaper plant (merit-order effect).
2. 100: the clocks go back, so the day has 25 hours x 4 quarter-hours.

3. Day-ahead products became 15-minute (96 per day). A plain row average would weight a quarter-hour like a full hour, so averages must be time-weighted.
4. UTC has no gaps or repeats, so every interval is unique; local time is ambiguous around DST changes.
5. On 28 May 2026 the floor was −500, so it is impossible: a data error. From 29 May 2026 the floor is −600, so it is a valid (extreme) price.
6. "You are rate limited; try again in 19 seconds." The client waits 20 s and retries, up to `max_retries`.
7. To keep the original evidence and make every result reproducible; corrections happen in code, in the processed layer.
8. The night-solar check (solar output at 00:00-03:00 local) and, for prices, the DST-aware completeness check.
9. Hourly average price < 0, versus at least one negative interval in the hour (or the total duration of negative intervals). They differ in quarter-hour data when an hour mixes negative and positive quarter-hours.
10. Comparing your figure with an independent published one. If it fails, find out whether your data or your definition differs before using the data.

---

9. Interview corner

Your 60-second story (STAR):

> *Situation:* I wanted to price renewable PPAs on real data. *Task:* first build a German price and generation dataset I could trust. *Action:* I wrote a small data platform in Python: an API client that respects rate limits, an immutable raw cache with provenance, DST-aware quality checks, and a reconciliation against the published 2025 average price and negative-hour counts. I handled the switch to 15-minute day-ahead products in October 2025 with time-weighted statistics. *Result:* the pipeline reproduces the published figures (see section 6), runs in CI with 52 offline tests, and caught two real issues: a pandas 3 behaviour change and an API rate limit.

Likely questions and strong answers:

- "*How do you deal with daylight saving in energy data?*" Store UTC, interval-beginning timestamps; group by local delivery day; check 23/24/25-hour completeness; never use wall-clock arithmetic.
- "*How do you know your data is correct?*" Automated checks from market rules (price limits), the calendar (DST completeness) and physics (no solar at night), plus reconciliation against official figures.
- "*What changed with the 15-minute day-ahead market?*" 96 products per day since 1 Oct 2025, more price granularity and intra-hour shape, so averages must be time-weighted and PPA clauses must define their settlement interval.
- "*Why do negative prices matter for a solar PPA?*" They hit exactly when solar produces; depending on the contract, the producer may pay, curtail or lose support, so they lower the capture price and must be priced in.

---

10. How this maps to real work at Encavis

In this project	In a company
Energy-Charts API	Licensed EPEX/EEX feeds and vendor APIs; data contracts
Parquet files on disk	Central data warehouse or lakehouse, owned with IT
`ppa-lab fetch` run by hand	Scheduled jobs (e.g. Airflow) with monitoring and alerts
`ppa-lab check` exit code	Data-quality gates that block downstream pricing runs
`.env` file	A secrets vault
Reconciliation to DGS/netztransparenz	Reconciliation to exchange settlements and accounting
ADRs, changelog, CI	Merge requests, code review, release notes, model governance

The *thinking* is identical. That is the point of building it this way.

---

11. Your to-dos for Step 1

- □ Read sections 1-4 with notes; draw the merit-order example yourself.
- □ Run section 5 and compare your numbers with section 6.
- □ **Request the ENTSO-E token:** register at transparency.entsoe.eu, then e-mail transparency@entsoe.eu with the subject "Restful API access" and your account e-mail. It can take a few days, which is why we ask today.
- □ **Make the first commit and publish the repository** (commands below). In a real team nobody commits for you, and the commit history is part of your portfolio.
- ☑ GitHub handle `ankitpatel1661` set in the User-Agent and in your CV's `\githuburl`.
- □ Do exercises 1-3 (warm-up) today; 4-6 tomorrow morning.
- □ **German vocabulary of this step** (Anki): die Strombörse, die Day-Ahead-Auktion, die Viertelstunde, die Zeitumstellung (Sommerzeit/Winterzeit), negative Strompreise, die Gebotszone, die Marktkopplung, der Mindestpreis / Höchstpreis, die Datenqualität, der Abgleich (reconciliation).
- □ Write a stand-up update as you would in a team: "*Yesterday: built the market-data pipeline for DE-LU, 2023 to Sep 2026; it reconciles with the published 2025 figures. Today: capture rates (PPA-2). Blockers: waiting for the ENTSO-E token.*"

First commit, then publish with the GitHub CLI (`gh`), which creates the repository, adds it as `origin` and pushes in one step:

```
git add -A
git status                                  # review what goes in: no data, no secrets
git commit -m "feat: market-data foundation (PPA-1)"
gh repo create ppa-lab --public --source=. --remote=origin --push \
  --description "Renewable PPA pricing & risk engine for German wind and solar"
gh run watch                                # follow the CI run until it is green
```

---

12. Glossary

Term	Meaning
Bidding zone	Area with one market price (DE-LU)
SDAC	Single Day-Ahead Coupling: the coupled European day-ahead auction
MTU	Market Time Unit: the length of one traded interval (15 min since Oct 2025)
Merit order	Supply offers sorted by price; sets the clearing price
Clearing price	Uniform price paid to all accepted sellers in an interval
Cannibalisation	Renewables lowering the price in the hours they produce
Interval beginning	Convention that a timestamp marks the start of its interval
Raw / processed layer	Untouched source data vs cleaned, derived datasets
Manifest	Metadata file describing where and how data was obtained
Reconciliation	Comparing own results with an independent source
Contract test	A test that checks an external API still behaves as expected
Rate limit / HTTP 429	Server-imposed cap on request frequency

13. Sources

- Energy-Charts API documentation, Fraunhofer ISE: api.energy-charts.info
- EPEX SPOT news, "15-minute MTU in SDAC was implemented" (Oct 2025)
- NEMO Committee notice: harmonised minimum clearing price –600 EUR/MWh from trading day 28 May 2026
- DGS evaluation of netztransparenz.de market values 2025 (Jan 2026): average spot 8.932 ct/kWh; negative hours 2019-2025
- Encavis Annual Report 2024 (negative-price shutdowns in Spain and Finland)
- Solarspitzengesetz summary, Stiftung Umweltenergierecht (Sep 2025)